

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
import torch
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
```

```
import torch
from torch.utils.data import Dataset, DataLoader
from PIL import Image
import os
```

```
class CaptchaDataset(Dataset):
    def __init__(self, image_folder, transform=None):
        self.image_folder = image_folder
        self.transform = transform
        self.image_files = [os.path.join(image_folder, f)
                             for f in os.listdir(image_folder)
                             if f.endswith(('png', 'jpg', 'jpeg', 'bmp'))]

        self.labels = [f.split("_")[0] for f in os.listdir(image_folder)]
        self.class_to_idx = {cls: idx for idx, cls in enumerate(sorted(set(self.labels)))}

    def __len__(self):
        return len(self.image_files)

    def __getitem__(self, idx):
        img_path = self.image_files[idx]
        img = Image.open(img_path).convert('RGB')
        label_name = self.labels[idx].split("_")[0]
        label = self.class_to_idx[label_name]

        if self.transform:
            img = self.transform(img)

        return img, label
```

```
IMAGENET_MEAN = [0.5, 0.5, 0.5]
IMAGENET_STD = [0.5, 0.5, 0.5]
```

```
train_transform = transforms.Compose([
```

```

        transforms.Resize((224, 224)),
        transforms.RandomRotation(5),
        transforms.ColorJitter(0.3, 0.3),
        transforms.ToTensor(),
        transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD)
    ])

    val_transform = transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD)
    ])

    test_transform = val_transform

```

```

train_dataset = CaptchaDataset("/content/drive/MyDrive/precog_captcha/task1/train", transform=train_transform)
val_dataset   = CaptchaDataset("/content/drive/MyDrive/precog_captcha/task1/validation", transform=val_transform)
test_dataset  = CaptchaDataset("/content/drive/MyDrive/precog_captcha/task1/test", transform=test_transform)

```

```

from torch.utils.data import WeightedRandomSampler
from collections import Counter

labels = [train_dataset[i][1] for i in range(len(train_dataset))]
counts = Counter(labels)

weights = [1.0 / counts[l] for l in labels]
sampler = WeightedRandomSampler(weights, len(weights))

```

```

batch_size = 64
train_loader = DataLoader(train_dataset, batch_size, sampler=sampler)
val_loader   = DataLoader(val_dataset, batch_size, shuffle=False)
test_loader  = DataLoader(test_dataset, batch_size, shuffle=False)

```

```

print(f"Train batches: {len(train_loader)}")
print(f"Val batches: {len(val_loader)}")
print(f"Test batches: {len(test_loader)}")

```

```

Train batches: 36
Val batches: 13
Test batches: 11

```

```

images, labels = next(iter(train_loader))
print(images.shape, labels.shape)

```

```
torch.Size([64, 3, 224, 224]) torch.Size([64])
```

```
import matplotlib.pyplot as plt
import random
import numpy as np

def show_samples(dataset, num_samples=8):
    """Display sample images from dataset"""
    fig, axes = plt.subplots(2, 4, figsize=(12, 6))
    axes = axes.flatten()

    indices = random.sample(range(len(dataset)), num_samples)

    for idx, ax in zip(indices, axes):
        image, label = dataset[idx]

        image_np = image.numpy().transpose(1, 2, 0)
        mean = np.array([0.5, 0.5, 0.5])
        std = np.array([0.5, 0.5, 0.5])
        image_np = std * image_np + mean
        image_np = np.clip(image_np, 0, 1)
        ax.set_title(f"Class: {dataset.class_to_idx[dataset.labels[idx].split("_")[0]]}\nLabel: {label}")
        ax.imshow(image_np)
        ax.axis('off')

    plt.tight_layout()
    plt.show()

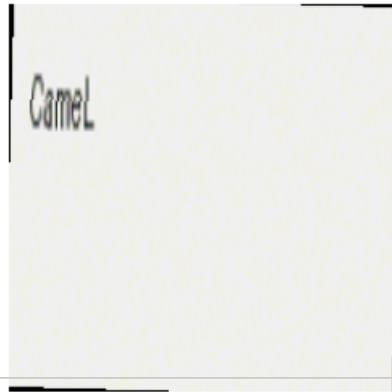
# Show samples from training set
print("\nSample training images:")
show_samples(train_dataset)
```

Sample training images:

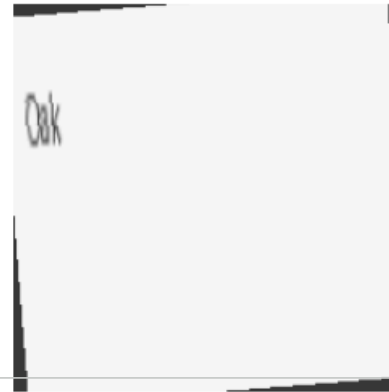
Class: 0
Label: 0



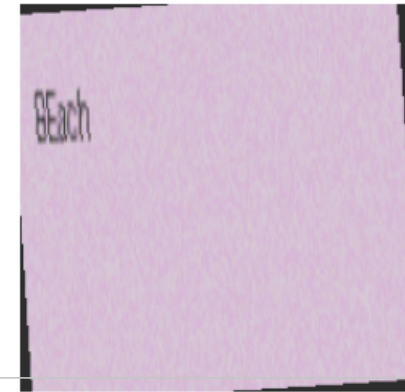
Class: 16
Label: 16



Class: 61
Label: 61



Class: 8
Label: 8



```
import numpy as np
from collections import Counter

def check_class_distribution(dataset):
    labels = [dataset[i][1] for i in range(len(dataset))]
    label_counts = Counter(labels)
    idx_to_class = {v: k for k, v in dataset.class_to_idx.items()}

    print(f"\nClass distribution for {dataset.image_folder}:")
    print(f"Total samples: {len(labels)}")
    print(f"Number of classes: {len(label_counts)}")
    counts = list(label_counts.values())
    print(f"Min samples per class: {min(counts)}")
    print(f"Max samples per class: {max(counts)}")
    print(f"Avg samples per class: {np.mean(counts):.1f}")
    print(f"Std samples per class: {np.std(counts):.1f}")
    print("\nFirst 10 classes:")
    for label_idx, count in list(label_counts.items())[:10]:
        class_name = idx_to_class[label_idx]
        print(f" {class_name} (idx {label_idx}): {count} samples")

    return label_counts

train_dist = check_class_distribution(train_dataset)
val_dist = check_class_distribution(val_dataset)
test_dist = check_class_distribution(test_dataset)
```

Class distribution for /content/drive/MyDrive/precog_captcha/task1/train:
Total samples: 2265
Number of classes: 72

Min samples per class: 4
Max samples per class: 200
Avg samples per class: 31.5
Std samples per class: 52.7

First 10 classes:

Banana (idx 5): 150 samples
Banyan (idx 6): 100 samples
Airport (idx 0): 200 samples
Beach (idx 8): 100 samples
Apple (idx 2): 200 samples
Ball (idx 4): 200 samples
Ambulance (idx 1): 200 samples
Auto (idx 3): 200 samples
Bat (idx 7): 100 samples
Bear (idx 9): 90 samples

Class distribution for /content/drive/MyDrive/precog_captcha/task1/validation:

Total samples: 825
Number of classes: 38
Min samples per class: 1
Max samples per class: 92
Avg samples per class: 21.7
Std samples per class: 25.1

First 10 classes:

Beach (idx 2): 50 samples
Banyan (idx 0): 92 samples
Bear (idx 3): 63 samples
Bat (idx 1): 83 samples
Bike (idx 4): 84 samples
Blocks (idx 5): 3 samples
Boat (idx 6): 88 samples
Buffalo (idx 7): 15 samples
Dice (idx 23): 14 samples
Goat (idx 33): 14 samples

Class distribution for /content/drive/MyDrive/precog_captcha/task1/test:

Total samples: 675
Number of classes: 38
Min samples per class: 3
Max samples per class: 112
Avg samples per class: 17.8
Std samples per class: 24.4

First 10 classes:

Banana (idx 0): 50 samples
Beach (idx 3): 50 samples
Bear (idx 4): 47 samples
Car-toy (idx 13): 6 samples
Goat (idx 34): 11 samples
Church (idx 16): 10 samples
Buffalo (idx 8): 23 samples
Elephant (idx 29): 10 samples

```
import torch
import torch.nn as nn
from torchvision.models import vit_b_16

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

num_classes = len(train_dataset.class_to_idx)

model = vit_b_16(weights="IMAGENET1K_V1")

model.heads.head = nn.Linear(model.heads.head.in_features, num_classes)

model = model.to(device)

print(model)
```

```

    )
    )
    (encoder_layer_11): EncoderBlock(
      (ln_1): LayerNorm((768,), eps=1e-06, elementwise_affine=True)
      (self_attention): MultiheadAttention(
        (out_proj): NonDynamicallyQuantizableLinear(in_features=768, out_features=768, bias=True)
      )
      (dropout): Dropout(p=0.0, inplace=False)
      (ln_2): LayerNorm((768,), eps=1e-06, elementwise_affine=True)
      (mlp): MLPBlock(
        (0): Linear(in_features=768, out_features=3072, bias=True)
        (1): GELU(approximate='none')
        (2): Dropout(p=0.0, inplace=False)
        (3): Linear(in_features=3072, out_features=768, bias=True)
        (4): Dropout(p=0.0, inplace=False)
      )
    )
  )
  (ln): LayerNorm((768,), eps=1e-06, elementwise_affine=True)
)
(heads): Sequential(
  (head): Linear(in_features=768, out_features=72, bias=True)
)
)

```

```

# Freeze all layers first
for param in model.parameters():
    param.requires_grad = False

# Unfreeze last transformer encoder blocks
for param in model.encoder.layers[-4:].parameters():
    param.requires_grad = True

# Unfreeze classification head
for param in model.heads.parameters():
    param.requires_grad = True

```

```

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(
    filter(lambda p: p.requires_grad, model.parameters()),
    lr=3e-4,
    weight_decay=1e-4
)

```

```

def train_one_epoch(model, loader):
    model.train()
    total_loss, correct = 0, 0

```

```

for images, labels in loader:
    images, labels = images.to(device), labels.to(device)

    optimizer.zero_grad()
    outputs = model(images)
    loss = criterion(outputs, labels)
    loss.backward()
    optimizer.step()

    total_loss += loss.item()
    correct += (outputs.argmax(1) == labels).sum().item()

acc = correct / len(loader.dataset)
return total_loss / len(loader), acc

def evaluate(model, loader):
    model.eval()
    correct = 0

    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            correct += (outputs.argmax(1) == labels).sum().item()

    return correct / len(loader.dataset)

```

EPOCHS = 40

```

for epoch in range(EPOCHS):
    train_loss, train_acc = train_one_epoch(model, train_loader)
    val_acc = evaluate(model, val_loader)

    print(f"Epoch [{epoch+1}/{EPOCHS}] "
          f"Loss: {train_loss:.4f} | "
          f"Train Acc: {train_acc:.4f} | "
          f"Val Acc: {val_acc:.4f}")

```

```

Epoch [1/40] Loss: 0.0420 | Train Acc: 0.9881 | Val Acc: 0.0000
Epoch [2/40] Loss: 0.0387 | Train Acc: 0.9876 | Val Acc: 0.0048
Epoch [3/40] Loss: 0.0259 | Train Acc: 0.9903 | Val Acc: 0.0036

```



```
test_acc = evaluate(model, test_loader)
print(f"Test Accuracy: {test_acc:.4f}")
```

Test Accuracy: 0.0000

```
train_acc=evaluate(model, train_loader)
print(f"Train Accuracy: {train_acc:.4f}")
```

Train Accuracy: 0.9916

```
loss_acc= evaluate(model, val_loader)
print(f"Val Accuracy: {loss_acc:.4f}")
```

Val Accuracy: 0.0000

```
from sklearn.metrics import confusion_matrix
import numpy as np

def get_all_preds(model, loader):
    model.eval()
    all_preds = []
    all_labels = []

    with torch.no_grad():
        for images, labels in loader:
            images = images.to(device)
            outputs = model(images)
            preds = torch.argmax(outputs, dim=1)

            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.numpy())

    return np.array(all_labels), np.array(all_preds)
```

```
y_true, y_pred = get_all_preds(model, test_loader)
```

```
cm = confusion_matrix(y_true, y_pred)
print("Confusion Matrix Shape:", cm.shape)
```

```
Confusion Matrix Shape: (46, 46)
```

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 8))
sns.heatmap(cm, cmap="Blues", xticklabels=False, yticklabels=False)
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix")
plt.show()
```

Confusion Matrix



```
class_accuracy = cm.diagonal() / cm.sum(axis=1)
print("Per-class Accuracy (first 10 classes):")
print(class_accuracy[:10])
```

```
Per-class Accuracy (first 10 classes):
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
/tmp/ipython-input-59994290.py:1: RuntimeWarning: invalid value encountered in divide
  class_accuracy = cm.diagonal() / cm.sum(axis=1)
```

Start coding or [generate](#) with AI.